Week 10 Milestone Report: Database and Query Defense

Course: CS3320 Database Systems, Instructor Dr. Chibuzo Ukegbu
Student: Shritej Gunda (individual project)
Project: Factory Sherlock: Manufacturing Simulation, Quality Control, and Defect Investigation on MySQL
Date: [submission date]
Repository: https://github.com/shritej6/factory-sherlock
Commit or tag: [commit hash after push]

1. Completion status, incomplete items, and blockers

Completed for this milestone:

- Revised ERD and normalization rationale (section 3; full version in docs/database/database-design.md).
- Schema and migrations: Alembic revision 3a5e982c3679 creates all 17 tables with primary keys, foreign keys with explicit delete behavior, UNIQUE and CHECK constraints, ENUM columns, and five secondary indexes; the downgrade drops the tables in dependency order.
- Seed data: reference data, demo accounts, and the demo scenario (seed 42) producing 9 orders, 35 batches, 700 units, 1400 inspections, 51 failed inspections, 12 machine events, and 2 alerts.
- Twelve non-trivial queries in database/queries with drafts and results (section 2).
- Initial CRUD evidence for the catalog and production areas (section 2).

Incomplete or in progress:

- The frontend is implemented (React 19, TypeScript, Vite 7, Tailwind 4, React Router 7, TanStack Query 5, Recharts 3) with screens for Login, Overview, Machines, Orders, Batches, Inspections, Traceability, Investigations list, Investigation detail (case board with evidence cards and show-the-SQL), Analytics (charts plus SQL Explorer), and Simulation. Nine screenshots were captured from the running application on 2026-11-03 (docs/media/screenshots); no video has been recorded.
- Automated tests and the GitHub Actions workflows exist: `uv run pytest` last reported 100 passed (95 percent coverage of the backend `app` package) and `pnpm test` 39 passed in 13 files; `.github/workflows/` holds `backend-ci.yml`, `frontend-ci.yml`, and `docs-lint.yml`. All three workflows passed on the first push to GitHub on 2026-11-04 (Backend CI run 41779775539), and the test matrix (docs/testing/test-matrix.md) records which scenarios are only partly covered.
- The Docker path (docker-compose.yml with scripts/setup/init_databases.sh mounted as the init script) has not been executed, because Docker is not installed on the development machine; the local MySQL path (scripts/development/local_mysql.sh) is the one that has been run.
- [List any item that was planned for Week 10 and is not finished.]

Blockers: [describe any blocker, or write "none"].

2. Required technical artifacts and repository commit or tag

Artifact	Location
Revised ERD	docs/database/database-design.md (Mermaid source also in docs/milestones/week08-proposal.md)
Migration	backend/migrations/versions/20261104_0106_create_manufacturing_schema.py
DDL snapshot	database/schema/schema.sql (generated by scripts/development/export_schema.sh)
Seed data	database/seeds/01_reference_data.sql, database/seeds/demo_scenario.json, backend/app/seed.py
Queries	database/queries/01 to 09 and database/queries/investigation/*.sql
API contract	docs/api/openapi.json (31 paths)
Repository commit or tag	[commit hash after push]

Query drafts and results. Each file carries a header with the business question, tables, techniques, parameters, and the index it relies on. Results are captured against the demo scenario (seed 42) loaded with uv run python -m app.seed --demo; paste the rows into the evidence column before submission.

Query	Business question	Techniques	Result evidence
01 machine_defect_rate_comparison	Which machines have the highest share of failed inspections in a window, against the fleet rate?	CTE, filtered LEFT JOIN, SUM OVER, RANK OVER, NULLIF	[paste the three result rows; CNC-03 is expected at rank 1 because of the injected tool wear]
02 production_performance	How far along is each order, how long did it take, and is it on time, late, or overdue?	Multi-table join, TIMESTAMPDIFF, CASE	[paste the first rows]
03 recurring_defects	Which defect categories recur per machine and what share does each represent?	Five-table join through two junction tables, SUM OVER PARTITION BY, DENSE_RANK	[paste rows for CNC-03]
04 material_lot_forward_trace	Which batches consumed a lot and how many resulting units failed?	Junction traversal, eight-table join, COUNT DISTINCT with CASE	[paste rows for one lot, for example L201]
05 unit_backward_trace	Where did a serial number come from and what did inspection find?	Join chain, correlated subquery with GROUP_CONCAT	[paste the row for one serial number]
06 quality_after_machine_events	How did the defect rate change after each machine event?	LATERAL derived tables, INTERVAL arithmetic	[paste rows for CNC-03]
07 order_completion_summary	Planned versus completed by product and status with subtotals?	GROUP BY WITH ROLLUP, GROUPING()	[paste the rollup rows]
08 batches_requiring_review	Which batches exceed a defect-rate threshold and are not quarantined or already under investigation?	CTE, EXISTS, optional parameter	[paste rows at threshold 0.10]
09 daily_production_trend	Units produced and failed per machine per day?	DATE bucketing, COUNT DISTINCT with CASE	[paste the three rows for one day and three machines]
investigation/defect_onset	In which hour did a machine's defect rate first rise?	DATE_FORMAT bucketing, LAG, cumulative SUM OVER	[paste rows for CNC-03]
investigation/machine_events_before_onset	What happened to the machine shortly before the onset?	Composite index range scan, TIMESTAMPDIFF	[paste rows; the TOOL_WEAR event is expected]
investigation/shared_material_lots	Do the affected batches share a material lot?	Expanding IN parameter, GROUP_CONCAT, correlated scalar subquery	[paste rows]

Constraints and indexes:

Constraint or index	Reason
CHECK quantity_completed <= quantity_planned (production_orders)	An order can never report more completed units than planned, regardless of the client
CHECK tolerance > 0, quantity > 0, quantity_used > 0, sequence_in_batch > 0	Zero or negative values would make the pass rule and traceability meaningless
CHECK window_end > window_start (investigations)	An empty or reversed window would make every rule return nothing
UNIQUE (unit_id, specification_id) on inspections	One measurement per unit per characteristic; the simulator and the API both rely on it

Constraint or index	Reason
UNIQUE (batch_id, sequence_in_batch) and UNIQUE serial_number	Stable identity for every unit; backward traceability starts from the serial
UNIQUE (investigation_id, sequence) on investigation_evidence	Evidence cards keep a stable order on the case board
ENUM status columns	Unknown statuses are rejected at the database boundary
ix_inspections_time_result (inspected_at, result)	Queries 01, 03, 06, 08 and the onset rule filter inspections by time window first
ix_machine_events_machine_time (machine_id, occurred_at)	Query 06 and the events-before-onset rule are range scans on one machine
ix_production_batches_machine_time (machine_id, started_at)	Query 08 scans batches for one machine in a window
ix_production_batches_order (production_order_id)	Query 02 aggregates batches per order
ix_production_orders_status (status)	Order list filters by status

Initial CRUD evidence to include in the PDF:

- [Capture of POST /api/v1/products and GET /api/v1/products/{id} as the admin or engineer account.]
- [Capture of PATCH /api/v1/products/{id} and the updated row in MySQL.]
- [Capture of POST /api/v1/production-orders, POST /api/v1/production-orders/{id}/status (PLANNED to RELEASED), and the rejected transition (COMPLETED to RELEASED) returning a 422 InvalidTransitionError.]
- [Capture of POST /api/v1/production-batches with material lots and the resulting batch_materials rows.]
- [Capture of a duplicate product code returning a 409 ConflictError.]

3. Decisions, alternatives, assumptions, constraints, and rationale

Normalization rationale. Every table has a single-column surrogate primary key except the three junction tables, which use the composite of their two foreign keys. Repeating groups are removed: specifications are rows of product_specifications, not columns on products; defects found by an inspection are rows of inspection_defects, not a delimited string. Partial dependencies are absent because each non-key column depends on the whole key, including in the junction tables (quantity_used depends on the batch and the lot together). Transitive dependencies are absent: machine name lives only on machines, product name only on products, and supplier only on material_lots. Two intentional, documented exceptions: inspections.result is stored rather than derived so historical results do not change when a tolerance is edited, and production_orders.quantity_completed is maintained by the service so order lists do not need an aggregate over batches.

Decision	Alternatives considered	Rationale
Composite primary keys on junction tables	Surrogate id plus a UNIQUE pair	The pair is the identity; the composite key doubles as the index the traversal queries need
Delete behavior per foreign key: RESTRICT for history, CASCADE for owned rows, SET NULL for optional references	ON DELETE CASCADE everywhere	Deleting a machine with recorded batches would silently destroy production history
Five secondary indexes only	Index every foreign key column	Each index is justified by a query in database/queries; production_units.produced_at is left unindexed and documented as a follow-up for larger datasets
Investigation evidence stores the SQL text and the result rows as JSON (a columns list plus value arrays, because MySQL reorders JSON object keys)	Store only a rule code and recompute on view	The case board shows the exact query behind every finding, and rows keep their shape and column order per rule
Query results tied to a demo scenario with a fixed seed	Random data on every run	Results are reproducible for the defense and for tests

Assumptions: the demo scenario covers one production day (06:00 to 12:00) on three machines; thresholds (elevated defect rate 5 percent and twice the fleet, batch review rate 10 percent, six hour event lookback) are documented in backend/app/investigation/rules.py and docs/investigation.

4. AI help: what, why, accepted, revised, rejected, and verification

Tool: Claude Code (Anthropic), used to draft the migration, the simulation engine, and query headers. The student ran migrations up and down, loaded the seed, and executed every query against MySQL 8.4 before keeping the code.

Stage	Task	Recommendation	Decision	Verification
Migration	Alembic downgrade	The generated downgrade dropped the secondary indexes before dropping the tables	Rejected: MySQL refuses to drop an index that backs a foreign key while the constraint still exists, so the explicit index drops were removed and the indexes are dropped with their tables	Confirmed by running alembic downgrade base and alembic upgrade head against MySQL 8.4
Simulation	End-of-run machine state	The first simulation engine moved a machine in WARNING straight to IDLE when the run finished	Revised: the shared state machine (domain/state_machines.py) rejected the transition because WARNING allows only RUNNING, STOPPED, or FAULT; the design was changed so a faulted machine stays in WARNING at the end of a run and only a healthy RUNNING machine returns to IDLE	Confirmed by running the demo scenario against MySQL 8.4: CNC-03 finishes in WARNING and CNC-01 in IDLE, so the unresolved fault remains visible in the machine status

Lesson recorded: the state machine is one shared table of allowed transitions, so a transition that the simulator takes must also be legal for the API; letting the domain layer reject the generated code was the intended safety net.

5. Lessons learned at this milestone

- A composite index on (machine_id, occurred_at) or (inspected_at, result) serves several queries at once when the leading column matches the most selective filter.
- Writing the business question and expected shape in the query header before the SQL kept every query tied to a screen or rule and made results easy to check.
- MySQL-specific behavior (index and foreign key coupling, LATERAL support from 8.0.14, GROUPING() with ROLLUP) must be verified on the real server, not assumed from general SQL knowledge.
- [Add one lesson in your own words.]

6. Support or skill development needed for the next stage

[State any instructor support or skill development you need for Week 12, or write "None at this time".]

7. Next-step plan (Week 12)

1. Extend the API test suite where the test matrix marks scenarios as partly covered (for example the 503 error body, the cascade and SET NULL foreign key behaviour, and quarantine rollback).
2. Produce the data-flow diagram for the investigation workflow (request, service, rule engine, queries, evidence rows).
3. Demonstrate a database-backed workflow (open an investigation on CNC-03) plus a controlled failure (an OPERATOR attempting to close an investigation receives 403).
4. Capture screenshots and videos from the running frontend, and run the Docker path and the GitHub Actions workflows to confirm them.
5. Keep docs/ai_evidence current with annotated conversation sequences.

8. Personal reflection (Shritej Gunda)

[Write 150 to 250 words in your own voice. Replace each prompt.]

[Which query did you find hardest to write or understand, and how did you confirm it returns the right rows?]

[Explain one normalization or indexing decision in your own words as you would in the defense.]

[Describe one AI suggestion from this milestone that you revised or rejected, how you found the problem, and what you learned about MySQL from it.]

[What do you need to practice before the Week 12 back end and security defense?]

9. Personal-work summary

[List the artifacts you personally produced or verified for this milestone, one line each, with the file path or screen name.]